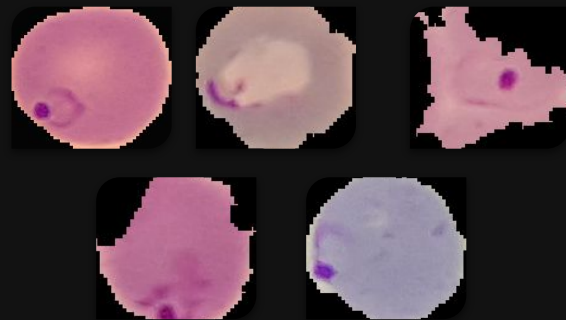


Malaria Detection

Advance Data Science – Deep
Learning Capstone Project



Aashit Soni

04.17.25

Project Definition

A traditional diagnosis and detection of Malaria is time consuming and tedious process.

It is dependent on experience and expertise of technician performing a task to analyze red blood cell image to determine if the sample is infected one or uninfected one.

Objective of this project is to automate – diagnosis process.

It will propose AI/ML based deep learning solution to classify red-blood cell image into infected/uninfected class.

There is clear set of images that are provided to us – In this project we will try to use different data exploration and manipulation technique to feed into number of different AI model to predict classification for the image.

The output of the project shall focus on classification using deep learning

It also shall recommend implementation details for deploying the solution.

Overview of Data

A compressed file is provided that has clearly marked classification folder for training and test data set.

Each dataset has labelled images for infected and parasitized red-blood cell image.

The images are in png format

Training

Total 24958

Parasitized : 12582

Uninfected: 12376

Shape : 64 x 64 x 3

Min value : 0

Max value : 255

The data seems balanced on both the classes

Test

Total: 2600

Parasitized: 1300

Uninfected: 1300

Shape : 64 x 64 x 3

Min value: 0

Max value: 255

The data seems balanced on both the classes

Data Pre-processing

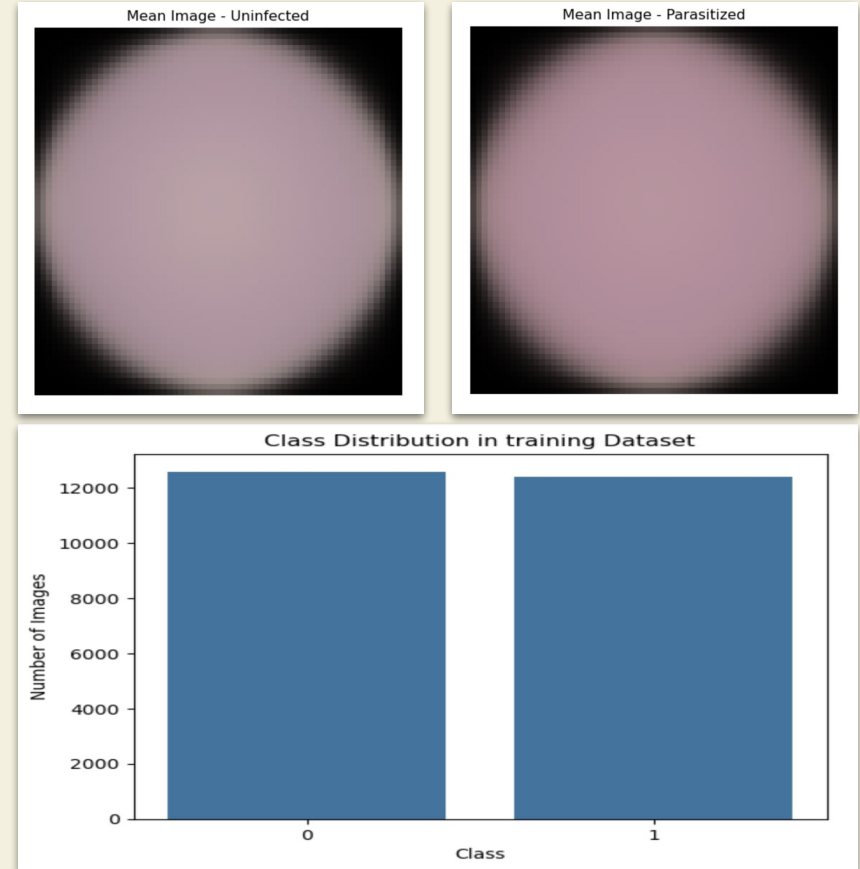
Creating tensorflow image data-set: provides access to training data and testing data with its label

It create 4D dataset - used in CNN model (includes #of image x 64 x 64 x 3)

Finding out maximum and minimum values to see the data spreading - even if it is pixels here.

Finding out average of the infected as well as uninfected images to see if that can provide any meaning information for our model building.

Normalization of image pixels - basically making pixels in range [0, 1] from [0, 255]



Data Exploration

Clear marking on the infected cells

Most of the uninfected seems clear

Both infected and uninfected cells have multiple colors

A mean image of infected and uninfected red blood shell

Converting RGB into HSV scale of images

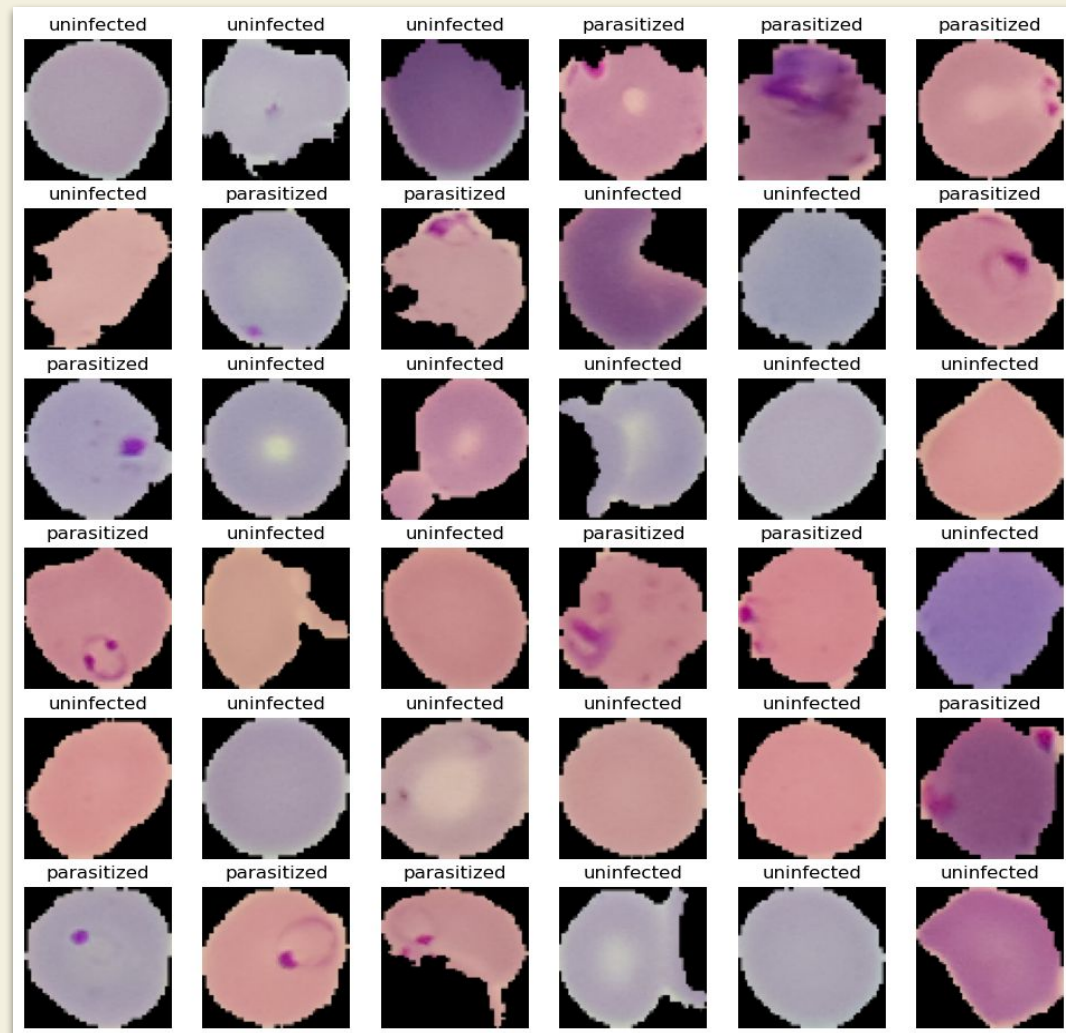
Gaussian blurring of images

Finding:

Blurring didn't provided any added value for the solution

Mean images are more blurred

HSV images are hard to analyse and make any inference



Modeling Techniques

Starting with base model – 2 or 3 convolutional layer

Evaluate a model adding few more convolutional layer, changing activation function

Having a image augmentation with either one of the tried model or having totally different evolved model

Using any pre-trained model e.g. vgg16

- with pre-trained weights and parameters

- With customized weights – (extreme slow on apple silicon)

Select a model :

- Has acceptable model performance (seconds per epochs with available h/w resource)

- Use confusion matrix – lowest false positive and false negative numbers

- Have best accuracy and lowest loss numbers

Base Model

Input of 64 x 64 x 3

Total of 3 CNN layer with ReLu activation

One flattening layer

One fully connected layer

Output layer with softmax activation

Total Parameters: 646,338

Non Trainable Parameter: 0

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_1 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_2 (Conv2D)	(None, 12, 12, 64)	36,928
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 64)	589,888
dense_1 (Dense)	(None, 2)	130

Total params: 646,338 (2.47 MB)

Trainable params: 646,338 (2.47 MB)

Non-trainable params: 0 (0.00 B)

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 64, 64, 3)	0	-
conv2d_6 (Conv2D)	(None, 64, 64, 32)	896	input_layer_2[0]...
batch_normalization (BatchNormalization)	(None, 64, 64, 32)	128	conv2d_6[0][0]
leaky_re_lu_2 (LeakyReLU)	(None, 64, 64, 32)	0	batch_normalizat...
max_pooling2d_4 (MaxPooling2D)	(None, 32, 32, 32)	0	leaky_re_lu_2[0]...
conv2d_7 (Conv2D)	(None, 32, 32, 64)	18,496	max_pooling2d_4[...
batch_normalization (BatchNormalization)	(None, 32, 32, 64)	256	conv2d_7[0][0]
activation_1 (Activation)	(None, 32, 32, 64)	0	batch_normalizat...
max_pooling2d_5 (MaxPooling2D)	(None, 16, 16, 64)	0	activation_1[0][...
conv2d_8 (Conv2D)	(None, 16, 16, 128)	73,856	max_pooling2d_5[...
batch_normalization (BatchNormalization)	(None, 16, 16, 128)	512	conv2d_8[0][0]
leaky_re_lu_3 (LeakyReLU)	(None, 16, 16, 128)	0	batch_normalizat...
conv2d_9 (Conv2D)	(None, 16, 16, 128)	147,584	leaky_re_lu_3[0]...
batch_normalization (BatchNormalization)	(None, 16, 16, 128)	512	conv2d_9[0][0]
leaky_re_lu_4 (LeakyReLU)	(None, 16, 16, 128)	0	batch_normalizat...
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0	leaky_re_lu_4[0]...
reshape (Reshape)	(None, 1, 1, 128)	0	global_average_p...
dense_4 (Dense)	(None, 1, 1, 16)	2,064	reshape[0][0]
dense_5 (Dense)	(None, 1, 1, 128)	2,176	dense_4[0][0]
multiply (Multiply)	(None, 16, 16, 128)	0	leaky_re_lu_4[0]... dense_5[0][0]
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0	multiply[0][0]
dense_6 (Dense)	(None, 128)	16,512	global_average_p...
dropout_1 (Dropout)	(None, 128)	0	dense_6[0][0]
dense_7 (Dense)	(None, 64)	8,256	dropout_1[0][0]
dropout_2 (Dropout)	(None, 64)	0	dense_7[0][0]
dense_8 (Dense)	(None, 2)	130	dropout_2[0][0]

Total params: 271,378 (1.04 MB)
Trainable params: 270,674 (1.03 MB)
Non-trainable params: 704 (2.75 KB)

Model 3

Adds few more convolutional layer on top of the base model.

Add squeeze and excite function to recalibrate the model performance for better accuracy.

Add batch normalization layer.

It changes the activation function

Total Parameters: 271,378

Non Trainable Parameters: 704

Model 5

Pre-trained vgg16 model

Global Average Pooling layer

Dense layer

Flatten Layer

Output layer with softmax activation

Total Parameters: 65,922

Non Trainable Parameter: 14,714,688

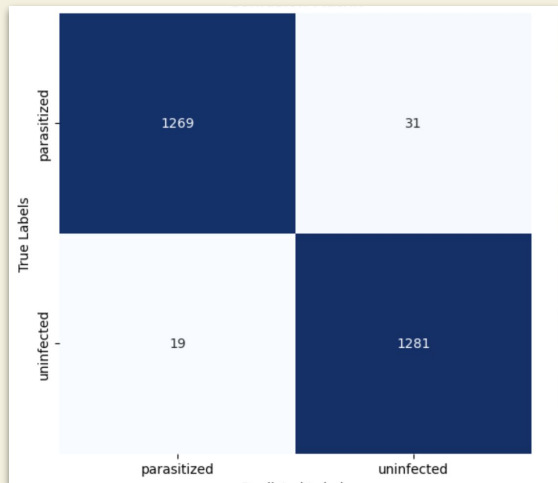
Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 2, 2, 512)	14,714,688
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None, 512)	0
dense_11 (Dense)	(None, 128)	65,664
dropout_4 (Dropout)	(None, 128)	0
dense_12 (Dense)	(None, 2)	258

Total params: 14,780,610 (56.38 MB)

Trainable params: 65,922 (257.51 KB)

Non-trainable params: 14,714,688 (56.13 MB)

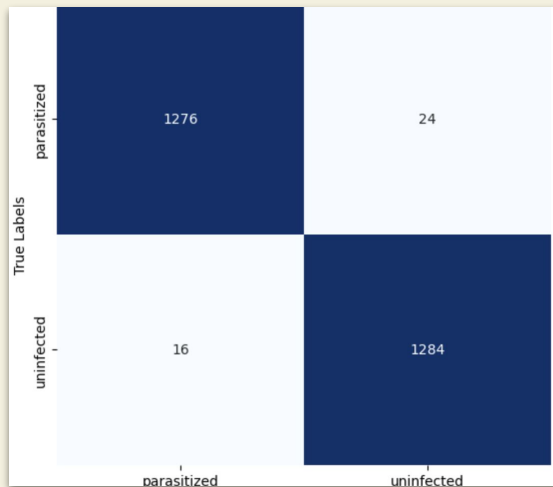
Confusion Matrix



Base Model

A total errors = 50 out of 2600

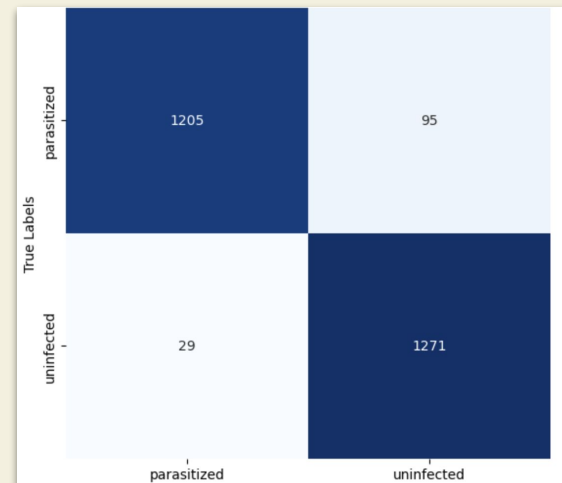
Decent model with optimistic numbers



Model 3

A lowest errors = 40 out of 2600

Little costly model but best performance

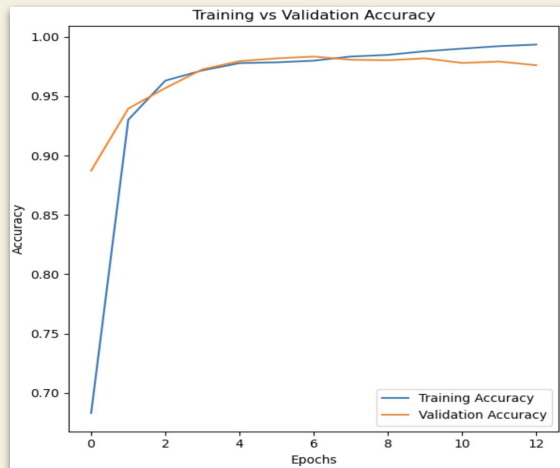


Model 5

Very high errors = 134 out of 2600

Slowest model, highest error rate

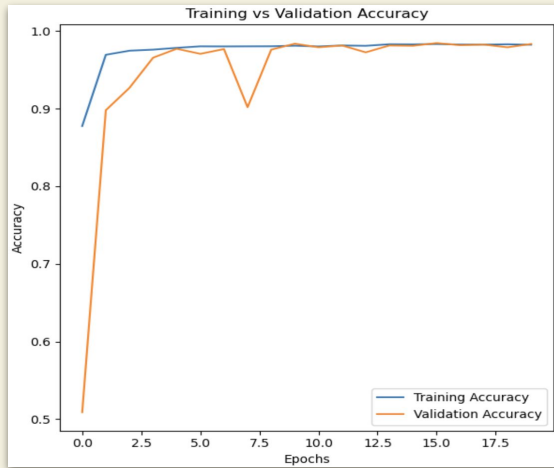
Accuracy



Base Model

Training: 0.9826

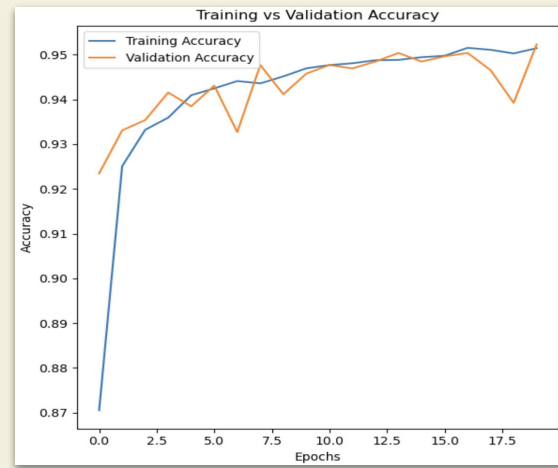
Test: 0.9838



Model 3

Training: 0.9812

Test: 0.9856

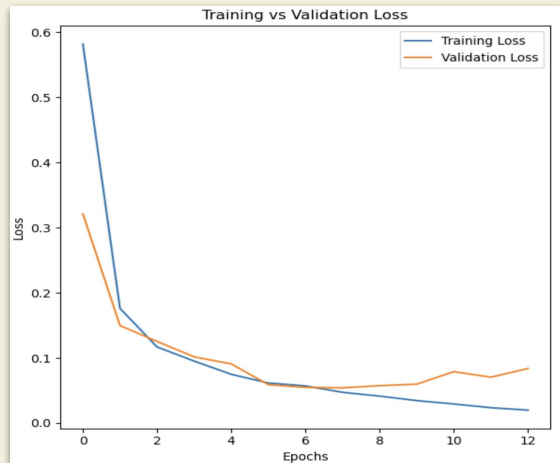


Model 5

Training: 0.9516

Test: 0.9580

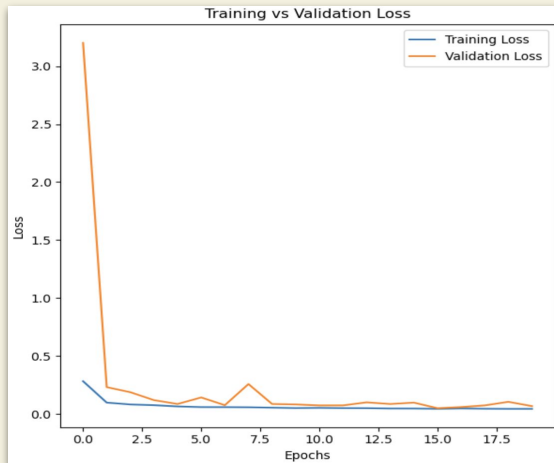
Loss variations



Base Model

Training: 0.0479

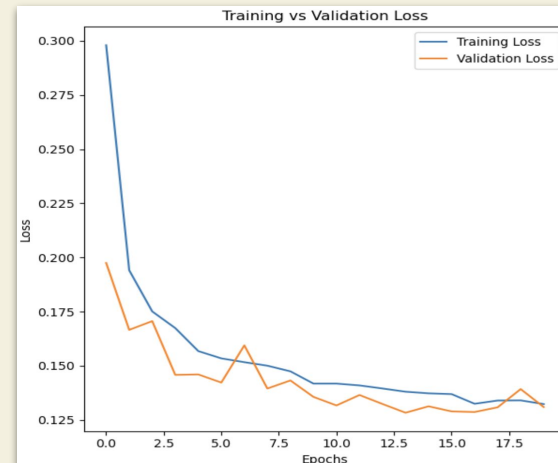
Test: 0.0488



Model 3

Training: 0.0623

Test: 0.0451



Model 5

Training: 0.1313

Test: 0.1239

Comparison

Base Model

Accuracy: 0.9838

Loss: 0.0488

False Positive: 19

False Negative: 31

Epoch runtime: 22 seconds

Errors: 40/2600

Model 3

Accuracy: 0.9856

Loss: 0.0451

False Positive: 16

False Negative: 24

Epoch runtime: 85 seconds

Errors: 40/2600

Model 5

Accuracy: 0.9580

Loss: 0.1239

False Positive: 29

False Negative: 95

Epoch runtime: 300 seconds

Errors: 134/2600

Final Solution – Model 3

AI/ML model implemented into server – that has compute power – does not need GPU/TPU – a cpu based server shall be fine.

Non-relational db storage e.g. mongoDB etc. that will organized data into test, train and unclassified DB.

A image validation engine that would make sure to validate the image and perform the data pre processing before it sort them out into unclassified DB.

A model – service process can classify the images stored into unclassified DB and generate the output.

A generated output can be disseminated to human – for verification and communicated to the respective user.

Once verified – image sorting server can distribute the image for future training and validation purpose.

A frequent processing for the model – that can be retrained and fit to improve the performance and accuracy

Implementation

